DevTools Web: Load, Capacity and Hardening Report

Oct 28, 2026 · @Golam Rabbi

Summary

DevTools Web serves roughly 500 to 1,300 actively working users per CPU core, slows down instead of crashing when overloaded, and is production-ready after four fixes made during this test.

- **Capacity:** about 90 requests per second per core for a typical mix of tools, and about 35 to 40 for a deliberately heavy mix with 5% Excel conversions.
- **Overload:** 150 simultaneous users on 2 CPU cores produced zero errors and no worker restarts in the final configuration. Responses slowed to 2 to 5 seconds.
- **Real weakness found:** with the original server setup, four stalled connections froze the site for everyone. This is fixed.
- **What to book:** 2 vCPU and 2 GB RAM for up to about 500 active users, 4 vCPU and 4 GB RAM for up to about 2,500. Beyond that, run several instances behind a load balancer.

Scope and method

The test answers three questions: how many users the app serves at once, whether it crashes under load, and how much hardware to book. All numbers come from one small machine, so treat them as estimates to re-measure on your own hardware.

Environment

- 2 vCPU and 8 GB RAM sandbox; the load generator ran on the same machine and took part of the CPU.
- Python 3.13, Flask 3.1, gunicorn 26, run as in production but with the testing config (CSRF on, rate limiting off) so capacity is measured, not 429 responses.

Four tests

1. **Service time per tool:** 8 to 25 sequential requests per case against one worker.
2. **Overload:** 150 simulated users with 50 to 300 ms think time for 20 to 25 seconds. The mix was 45% small JSON format, 25% page views, 15% cron, 10% 190 KB JSON and 5% Excel with 1,000 rows. It ran against three server setups: 3 sync workers (the original), 3 workers with 8 threads each, and the final configuration.

3. **Stalled clients:** 4 and 12 connections that announce a request body and never send it, followed by one normal request.
4. **Worst-case Excel:** the densest workbook the limits allow, 0.64 MB and 139,930 cells.

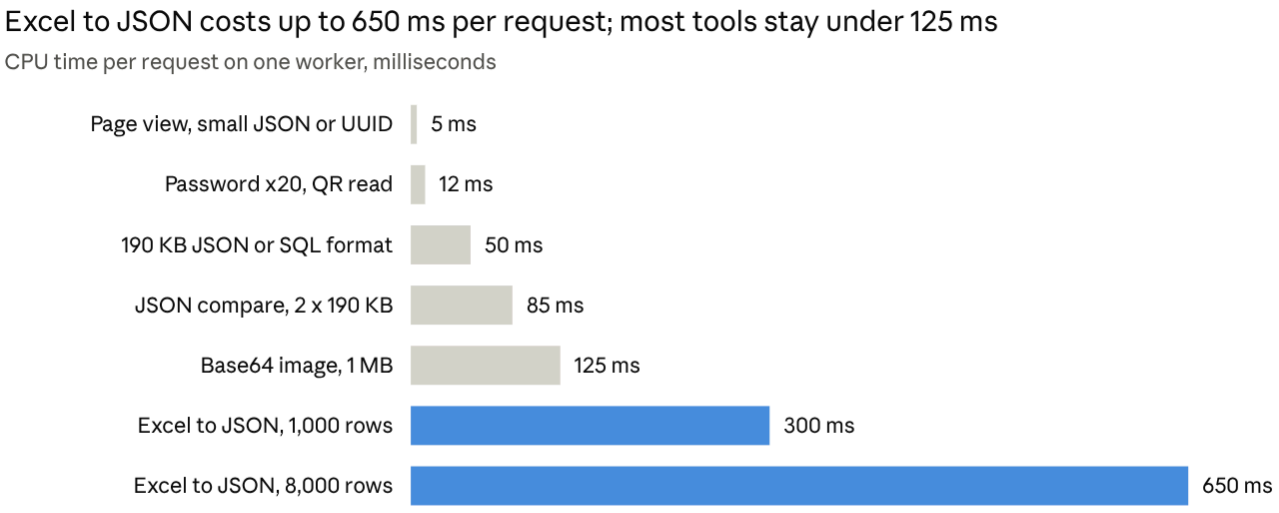
Limits of the test: shared CPU, a synthetic user model, no TLS or reverse proxy, no network latency, runs of under 30 seconds, and no multi-instance setup.

Observations

Light tools cost 3 to 15 ms of CPU per request, image and Excel tools cost 125 to 640 ms, and CPU rather than memory is what runs out first.

Speed per tool

The median CPU time per request on one worker is charted below. A page view or small JSON format takes about 3 ms, while Excel to JSON is roughly 100 times heavier. The densest workbook the limits allow took 2.6 seconds, almost all of it parsing; the security screening took 35 ms.



scripts/loadtest.py and bench runs, 2 shared vCPU · midpoints of measured ranges

Throughput

- The test mix costs about 24 ms per request, which is about 41 requests per second per core. The three sync workers measured 52 requests per second on 2 shared cores.
- With 1% Excel instead of 5%, the cost falls to about 11 ms, or roughly 90 requests per second per core.
- Assuming each user sends one request every 15 seconds, one core serves about 500 users with the heavy mix and about 1,300 with the typical mix.

Behaviour under overload

150 simulated users on 2 cores, three server setups:

Setup	Requests	Throughput (req/s)	Errors	Median latency	Peak memory (MB)
3 sync workers (original)	1,385	52	0	2.2 to 4.6 s	262
3 workers x 8 threads	1,073	41	10 connection errors	2.2 to 6.3 s	361
Final configuration	1,019	47	0	2.2 to 4.7 s	216

Every completed request returned HTTP 200, the master process stayed alive, and no worker timed out. The 10 connection errors in the middle run did not recur in the final configuration; I did not isolate their cause, which is most likely the single-process load generator.

Stalled clients

- **Original setup:** 4 stalled connections left a normal request without a response for 8 seconds, so the whole site was frozen.
- **Threaded workers:** with 4 and with 12 stalled connections, a normal request was answered at once with HTTP 200.

Memory

- An idle worker uses about 79 MB, and 109 MB after heavy uploads.
- All workers together peaked at 216 to 361 MB under load.
- A 20-megapixel image, the pixel cap, could briefly need about 250 MB. This is an estimate, not a measurement.

Risks found

The app does not crash under load, but six weaknesses could hurt availability or fairness in production.

#	Risk	Evidence	Severity
1	Stalled clients freeze plain sync workers	4 stalled connections left a normal request unanswered for 8 s	High

#	Risk	Evidence	Severity
2	Rate limit shared by all visitors behind a proxy	Every request appears to come from the proxy's IP, so one bucket serves everyone	High
3	Rate limits are kept per worker in memory	Limits multiply with the worker count and reset on restart	Medium
4	Excel conversions can starve other tools	A dense 1 MB workbook costs up to 2.6 s of CPU	Medium
5	Oversized bodies were parsed before the per-tool limit applied	The global 6 MB ceiling was the only early guard	Medium
6	Large images can spike memory	A 20-megapixel image may need about 250 MB (estimate)	Low

Under overload, requests queue and slow to seconds rather than fail. There is no automatic scaling, so sustained overload means slow pages until capacity is added.

Solutions implemented

Each risk has a fix in the code, and all 101 automated tests pass.

#	Fix	Where
1	Threaded workers (gthread, 4 threads per worker), 30 s timeout, workers recycled every ~1,000 requests. 12 stalled clients no longer blocked normal requests	`gunicorn.conf.py`
2	`PROXY_HOPS` makes the app trust exactly that many proxies' forwarded headers, so each visitor gets their own rate-limit bucket. Verified by a unit test only, not behind a real proxy	`app/__init__.py`, `app/config.py`
3	Shared limits through `RATELIMIT_STORAGE_URI=redis://...` for more than one worker or container	`app/config.py`
4	Excel runs through a semaphore (`EXCEL_MAX_PARALLEL`, default 2) and returns a polite "busy" message when full, plus a 10 per minute limit, a 10 s time budget and row, column and cell caps	`app/tools/excel_json.py`

#	Fix	Where
5	A per-tool body limit runs before CSRF and form parsing, so an oversized upload is refused with HTTP 413 early	`app/__init__.py`, `app/uploads.py`
6	A 20-megapixel cap on images, and uploads kept in memory and bounded by the per-tool limits	`app/config.py`, `app/uploads.py`

Also added: a `/healthz` probe exempt from rate limits, an access log without query strings, and a non-root container with a health check.

Upload screening

Every upload is checked before any tool sees it. Files are never executed, written to disk or served back.

- **Size limits:** Excel 1 MB, Base64 image 5 MB, certificate 2 MB, QR code 2 MB. They are configurable through environment variables, with a hard ceiling of 20 MB.
- **Excel:** rejects macros, embedded objects, external links, DTD and entity declarations, path traversal, zip bombs and legacy `.xls`.
- **Images:** the magic bytes must match, nothing may follow the image data, and the image must decode fully.
- **Certificates:** private keys are refused.
- **Any file type:** executables, scripts, PDFs, archives and HTML or SVG are recognised and refused with a specific reason.

Sizing and configuration

For up to about 500 active users, book 2 vCPU and 2 GB RAM, then scale CPU first.

Expected active users	Book	Notes
Up to ~500 (team or small public)	2 vCPU, 2 GB RAM	1 instance
~500 to 2,500	4 vCPU, 4 GB RAM	1 instance, `WEB_CONCURRENCY=4`
More, or high availability	2 to 3 instances of 2 vCPU, 2 GB RAM	Load balancer plus Redis for shared rate limits

These figures assume each user sends one request every 15 seconds, and were measured on 2 shared vCPUs with the load generator on the same machine. Re-measure on your own hardware with `scripts/loadtest.py`.

Settings

Variable	Default	Purpose
`SECRET_KEY`	none	Required in production, at least 32 characters
`WEB_CONCURRENCY`	CPU cores (2 to 8)	Worker processes
`WEB_THREADS`	4	Threads per worker
`PROXY_HOPS`	0	Set to 1 behind nginx or a load balancer
`RATELIMIT_STORAGE_URI`	`memory://`	Use Redis when running more than one worker or instance
`EXCEL_MAX_PARALLEL`	2	Concurrent Excel conversions per process
`UPLOAD_MB_EXCEL_JSON`, `UPLOAD_MB_BASE64_IMAGE`, `UPLOAD_MB_CERTIFICATE`, `UPLOAD_MB_QR_CODE`	1, 5, 2, 2	Upload limits in MB, capped at 20

Put nginx or a CDN in front for TLS, static files and request buffering. Nothing is stored, so disk needs are tiny.

Remaining risks and next steps

The app is ready for a small to medium deployment, with four open items to close before heavy public use.

Not covered

- **Real-network testing:** all load was generated from the same machine, so network latency and TLS cost are not included.
- **Behind a real proxy:** `PROXY_HOPS` is verified by a unit test, not through nginx or a cloud load balancer.
- **Redis:** shared rate limiting is configured but was not load-tested.

- **Long runs:** tests lasted minutes, so slow memory growth over days is not ruled out; worker recycling limits it.
- **Antivirus:** screening is structural (file type, content and size checks). It does not replace a malware scanner if files are ever stored.

Next steps

1. Deploy behind nginx or the cloud load balancer with `PROXY_HOPS=1`, and confirm two different visitors get separate rate-limit buckets.
2. Run `scripts/loadtest.py` on the production-sized machine and compare it with these figures.
3. Add Redis before running more than one worker or instance.
4. Alert on HTTP 429 and 5xx rates, worker restarts and memory per worker.
5. Re-run `pytest`, `bandit` and `pip-audit` on every release.

Appendix: checks run

Check	Result
`pytest`	101 tests pass, including hostile-file and capacity tests
`pip-audit`	No known vulnerable dependencies
`bandit`	2 findings, both false positives (the test key and an empty default secret)
Browser check (Chromium)	Strict content security policy held, with no inline script or style